DBMS FINAL PROJECT – DAY 1

System Identification, Database Planning, and GitHub Repository Setup

Subject: Database Management Systems
Program/Section: BSIT 3 (BSIT3-T2)
Group: Group 1
Academic Year: 1st Sem, AY 2026-2027
Members: Excel Jim Molina · Jezreel C. Talaid (Group Leader) · Randolf Comonlay · Jasper Patagnan
Repository: https://github.com/USERNAME/DBMS-FINAL-2026-GROUP1

Task 1 – System Identification

Item	Details
Proposed system title	**UtangTrack: Sari-Sari Store Utang (Credit) Tracking Database System**
Organization / intended environment	Small neighborhood sari-sari stores in a barangay/community, where the owner regularly sells goods on credit ("utang") to trusted neighbors.
System description	A database-driven system that records customers, products, credit purchases, partial and full payments, due dates, and running balances for a sari-sari store. It lets the owner instantly see who owes how much, what was bought, what has been paid, and which accounts are overdue, and produces reports for store decisions.
Existing process	Credit sales are written by hand in a notebook ("listahan") or remembered by the owner. Each customer has a page or line where items and amounts are listed; payments are crossed out or written beside the entry. Totals are computed manually with a calculator.
Problems encountered	Lost, damaged or unreadable notebooks; arithmetic mistakes; disputes over what was borrowed or paid; no reminder of due dates; difficulty knowing the total money tied up in unpaid credit; no sales or stock records; slow lookup of a customer's balance.
Why the system is necessary	It replaces the notebook with a reliable, searchable and accurate record. It reduces lost income from forgotten or disputed debts, speeds up service, improves cash-flow monitoring, and gives the owner reports that support restocking and credit-limit decisions.

Task 2 – Problem Identification

#	Problem	What is happening	Who is affected	Consequences	How the database system addresses it
1	Loss and inaccuracy of handwritten records	Notebook entries get lost, torn, smudged, or miscomputed.	Store owner, customers	Lost income, wrong balances, arguments over amounts.	Centralized, validated storage with automatic balance computation and regular backups; every transaction has a date, items and amount.

2	No tracking of due dates and overdue accounts	The owner relies on memory to remember who owes and for how long.	Store owner	Late or never-collected debts; cash shortage for restocking.	Stores due dates and payment status; queries list overdue customers and aging of debts for follow-up.
3	Slow and difficult retrieval of customer balances	Searching pages of the notebook to find a customer's total utang.	Owner, helper, waiting customers	Slow service, lost sales, customer frustration.	Indexed customer records and a running-balance view return a customer's full credit history and balance instantly.
4	No reports or insight on credit and stock	Nothing summarizes total unpaid credit, top borrowers, or fast-moving items bought on credit.	Store owner	Poor credit decisions and over/under-stocking.	Report queries and views summarize total receivables, payments collected, credit per customer, and product usage.

Task 3 – Project Objectives

General Objective

To design and implement a reliable relational database system that accurately records, organizes and reports customer credit transactions, payments and balances for a sari-sari store, replacing the manual notebook process.

Specific Objectives

- To design a normalized relational database (at least 3NF) that stores customers, products, credit transactions, payments and users with no duplicated or inconsistent data.
- To automatically compute each customer's outstanding balance after every credit purchase or payment, with 100% consistency between transactions and balances.
- To enable retrieval of any customer's balance and transaction history in a single query, replacing manual notebook searching.
- To identify overdue accounts through due-date tracking and produce reports on total receivables, payments collected and top borrowers.
- To implement role-based access (owner, helper, customer) and maintain an audit log of changes to protect data integrity.
- To use GitHub to document and version-control all project files, with verifiable contributions from every member.

Task 4 – Target Users

User Type	Primary Responsibilities	Access Level / Permissions
Store Owner (Administrator)	Manages the store; approves credit, sets credit limits and due dates; records payments; manages products, stock and users; reviews reports.	Full access: create, read, update, delete on all tables; manage user accounts; view all reports; view audit logs.

Store Helper / Cashier (Staff)	Records credit purchases and payments during sales; looks up customer balances; updates stock when items are sold or received.	Limited access: add and read customers, products, credit transactions and payments; no deletion; no user management or audit logs.
Customer (Borrower)	Views own credit balance, itemized purchases, payments and due dates; confirms records are correct.	Read-only access limited to the customer's own records.
Database Administrator (Developer)	Maintains the database, backups, indexes and security; resolves technical problems.	Technical access to the database server; schema and backup management.

Task 5 – Initial Database Planning

Only entities and attributes are identified at this stage; the ERD will be done in a later phase.

Entity Name	Business Purpose	Information to be Stored (Key Attributes)
Customer	Stores the people who buy goods on credit.	customer_id, first_name, last_name, address, contact_number, credit_limit, date_registered, status
User	Stores system accounts for the owner, staff and customers who log in.	user_id, username, password_hash, role, full_name, customer_id (if borrower), is_active, created_at
Product	Stores items the store sells.	product_id, category_id, product_name, unit_price, unit, stock_quantity, reorder_level
Category	Groups products for organization and reports.	category_id, category_name, description
Utang (Credit Transaction)	Records each credit purchase made by a customer.	utang_id, customer_id, user_id, transaction_date, due_date, total_amount, status (unpaid/partial/paid), remarks
Utang Item (Transaction Detail)	Records the individual products and quantities in a credit transaction.	utang_item_id, utang_id, product_id, quantity, unit_price, subtotal
Payment	Records partial or full payments made by customers toward their utang.	payment_id, customer_id, utang_id, user_id, payment_date, amount_paid, payment_method, reference_note
Supplier	Stores the vendors from whom the store buys stock.	supplier_id, supplier_name, contact_person, contact_number, address
Stock Movement	Tracks stock coming in and going out of inventory.	movement_id, product_id, supplier_id, movement_type (in/out/adjust), quantity, movement_date, user_id
Reminder	Tracks follow-ups for due and overdue accounts.	reminder_id, customer_id, utang_id, reminder_date, message, status

Audit Log	Records changes made by users for accountability and security.	log_id, user_id, action, table_name, record_id, action_timestamp

Task 6 – DBMS Selection

Item	Details
Database name & classification	**MySQL** (MariaDB-compatible) – Relational (SQL) DBMS
Justification	The data is structured and strongly related (customers → credit transactions → items → payments), which fits a relational model with primary/foreign keys. MySQL is free, widely used, lightweight enough for a small store, supports transactions (InnoDB), stored procedures, functions, triggers and views for business logic, and integrates easily with REST APIs and web/mobile apps.
Advantages	Free and open source; ACID-compliant with InnoDB; strong referential integrity; stored procedures, triggers and views; large community and documentation; easy to set up with XAMPP/phpMyAdmin; fast for read-heavy lookups.
Limitations	Less suited to unstructured data; horizontal scaling is harder than with NoSQL; some advanced features (e.g., certain analytics functions) are fewer than in PostgreSQL; needs a defined schema upfront.
Management & modeling tools	MySQL Workbench (administration and ERD modeling), phpMyAdmin / XAMPP (local server), draw.io (diagrams), Postman (API testing), Git & GitHub (version control), Visual Studio Code (editing SQL).

GitHub Repository Setup

Item	Details
Repository name	DBMS-FINAL-2026-GROUP1 (public)
Initialized with	README.md and .gitignore
Collaborators (write access)	Excel Jim Molina, Jezreel C. Talaid, Randolf Comonlay, Jasper Patagnan
Folders	day-01/, day-02/, day-03/, database-design/, database/, screenshots/, progress-reports/

Group Members and Assigned Roles

Member	Role
Jezreel C. Talaid	Group Leader / Project Manager; GitHub repository owner; Google Classroom submission
Excel Jim Molina	Database Designer (entities, ERD, normalization)
Randolf Comonlay	Database Developer (SQL scripts, functions, stored procedures)
Jasper Patagnan	Documentation and Testing (reports, test data, screenshots)